



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

LIST OF EXPERIMENTS

1. Perform basic Image Handling and processing operations on the image is to read an image in python and Convert an Image to Gray-scale. `import cv2 import matplotlib.pyplot as plt` # Step 1: Read the image using OpenCV `image = cv2.imread('image.jpg')` # Replace with your image path

Check if image is loaded if image is

None: `print("Error: Image not`

`found!") else: print("Image loaded`

`successfully")`

Step 2: Convert BGR → RGB (for correct display)

`image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)`

Step 3: Convert to grayscale `gray =`

`cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)`

Step 4: Display images using matplotlib

`plt.figure(figsize=(8,4)) plt.subplot(1,2,1)`

`plt.imshow(image_rgb) plt.title("Original`

`Image") plt.axis('off')`

`plt.subplot(1,2,2)`

`plt.imshow(gray, cmap='gray')`

`plt.title("Grayscale Image")`

`plt.axis('off') plt.show()`

Step 5: Save grayscale image using matplotlib

`plt.imsave('grayscale_matplotlib.png', gray, cmap='gray')`

`print("Grayscale image saved successfully using matplotlib")`

Original Image



Grayscale Image



2. Perform basic Image Handling and processing operations on the image is to read an image in python and Convert an Image to Blur using Gaussian Blur

```
import cv2
import matplotlib.pyplot
as plt
```

```
# Step 1: Read Image (Handling) image = cv2.imread('image.jpg')
```

```
# Replace with your image path if image is None:
```

```
    print("Error: Image not found!") else:
```

```
    print("Image loaded successfully")    # Step 2: Convert BGR
```

```
→ RGB    image_rgb = cv2.cvtColor(image,
```

```
cv2.COLOR_BGR2RGB)
```

```
    # Step 3: Apply Gaussian Blur (Preprocessing)
```

```
    # (kernel size must be odd: (5,5), (7,7), etc.)
```

```
    blurred = cv2.GaussianBlur(image, (5, 5), 0)    # Convert
```

```
blurred image for display    blurred_rgb = cv2.cvtColor(blurred,
```

```
cv2.COLOR_BGR2RGB)
```

```
    # Step 4: Display Images
```

```
plt.figure(figsize=(8,4))    plt.subplot(1,2,1)
```

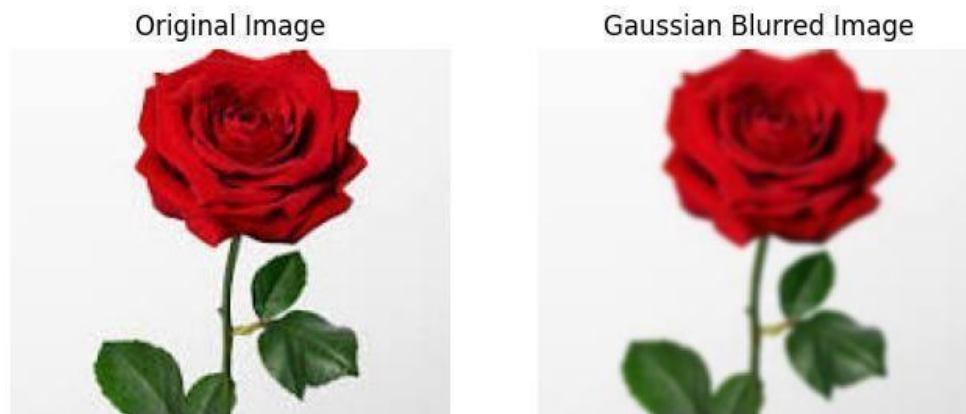
```
plt.imshow(image_rgb)    plt.title("Original
```

```
Image")    plt.axis('off')
```

```
plt.subplot(1,2,2)
```

```
plt.imshow(blurred_rgb)
plt.title("Gaussian Blurred Image")
plt.axis('off') plt.show()
# Step 5: Save Blurred Image
plt.imwrite('gaussian_blur_output.png', blurred_rgb)
print("Blurred image saved successfully")
```

OUTPUT:-



3. Perform basic Image Handling and processing operations on the image is to read an image in python and Convert an Image to show outline using Canny function.

```
import cv2 import matplotlib.pyplot
as plt
# Step 1: Read Image (Handling)
# ----- image = cv2.imread('image.jpg') #
Replace with your image path if image is None:
print("Error: Image not found!") else: print("Image loaded
successfully") # Step 2: Convert BGR → RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
# Step 3: Convert to Grayscale gray =
cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```

# Step 4: Apply Canny Edge Detection
# threshold1 = 100, threshold2 = 200
edges = cv2.Canny(gray, 100, 200) #
Step 5: Display Images
plt.figure(figsize=(10,4))
plt.subplot(1,3,1)
plt.imshow(image_rgb)
plt.title("Original Image") plt.axis('off')
plt.subplot(1,3,2) plt.imshow(gray,
cmap='gray') plt.title("Grayscale
Image") plt.axis('off')
plt.subplot(1,3,3) plt.imshow(edges,
cmap='gray') plt.title("Canny Edge
Output")
plt.axis('off') plt.show()
# Step 6: Save Edge Image plt.imsave('canny_edges_output.png',
edges, cmap='gray') print("Edge-detected image saved
successfully")

```



4. Implement histogram equalization on the given image and compare it with the original image using Open CV.

```

import cv2 import matplotlib.pyplot as plt # Step 1: Read Image
image = cv2.imread('image.jpg') # Replace with your image path
if image is None:    print("Error: Image not found!") else:
print("Image loaded successfully")

    # Step 2: Convert BGR → RGB (for display)  image_rgb =
cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

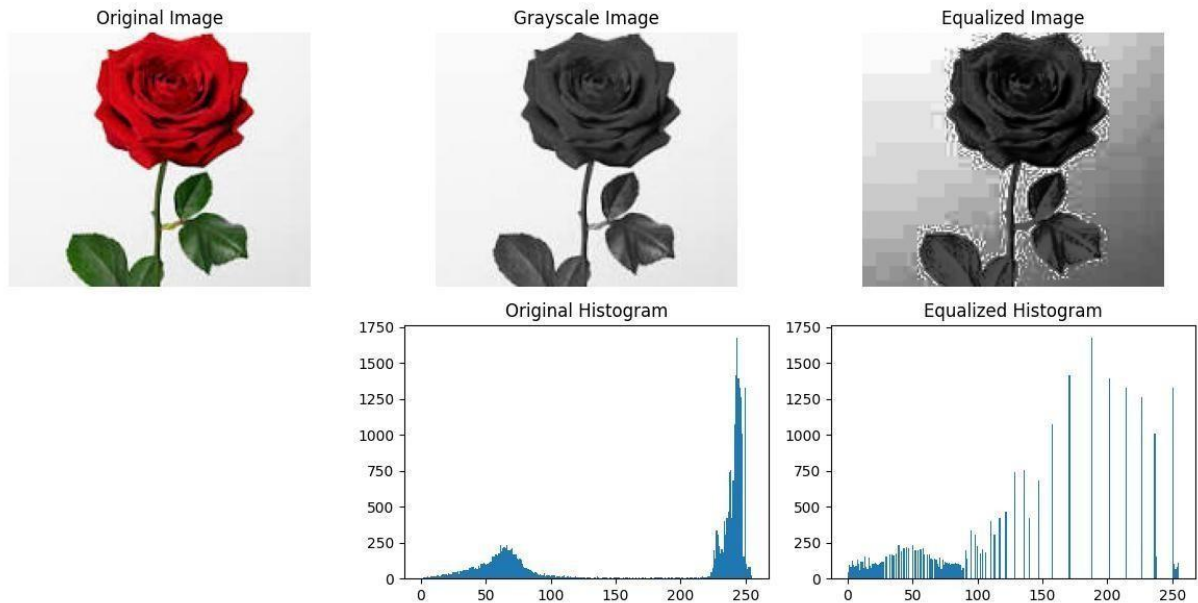
    # Step 3: Convert to Grayscale  gray =
cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # Step 4: Histogram Equalization  equalized
= cv2.equalizeHist(gray)
# Step 5: Display Images
plt.figure(figsize=(12,6))
plt.subplot(2,3,1)
plt.imshow(image_rgb)
plt.title("Original Image")  plt.axis('off')
plt.subplot(2,3,2)  plt.imshow(gray,
cmap='gray')  plt.title("Grayscale
Image")  plt.axis('off')
plt.subplot(2,3,3)
plt.imshow(equalized, cmap='gray')
plt.title("Equalized Image")
plt.axis('off')

    # Step 6: Plot Histograms
plt.subplot(2,3,5)  plt.hist(gray.ravel(),
bins=256)  plt.title("Original
Histogram")  plt.subplot(2,3,6)
plt.hist(equalized.ravel(), bins=256)
plt.title("Equalized Histogram")
plt.tight_layout()  plt.show()

```

```
# Step 7: Save Images  plt.imsave('original_rgb.png',
image_rgb)  plt.imsave('grayscale.png', gray,
cmap='gray')  plt.imsave('equalized.png', equalized,
cmap='gray')  print("All images saved successfully")
```

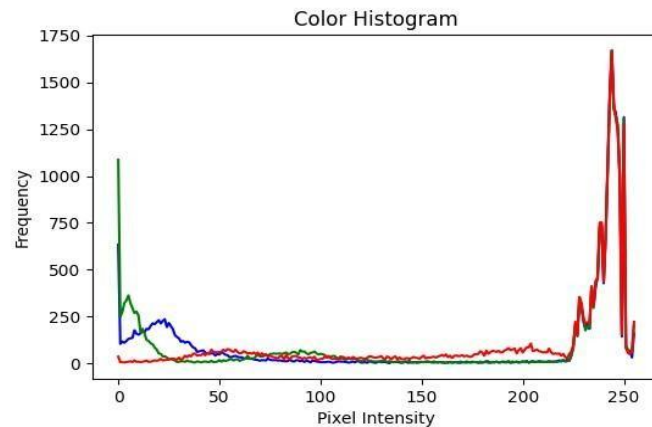


5. Write a Python function to analyze the histogram of the given input image based on color levels using Open CV.

```
import cv2 import matplotlib.pyplot
as plt def
analyze_histogram(image_path):
# Step 1: Read image  image =
cv2.imread(image_path)  if image
is None:
    print("Error: Image not found!")    return
# Step 2: Split into color channels (BGR)
channels = cv2.split(image)

colors = ('b', 'g', 'r')  plt.figure(figsize=(6,4))
plt.title("Color Histogram")  plt.xlabel("Pixel
Intensity")  plt.ylabel("Frequency")
```

```
# Step 3: Calculate histogram for each channel    for
channel, color in zip(channels, colors):
    hist = cv2.calcHist([channel], [0], None, [256], [0,256])    plt.plot(hist,
color=color)    plt.show() # Call the function analyze_histogram('image.jpg')
```



6. Perform basic Image Handling and processing operations on the image is to read an image in python and Convert an Image to erode using Erode function.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

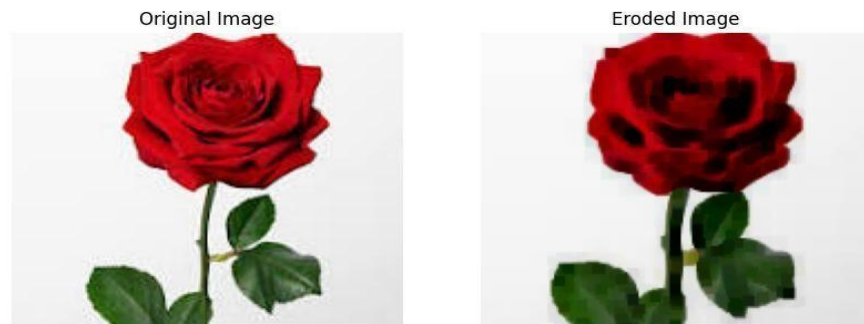
# Read image
image = cv2.imread('image.jpg')
image_rgb = cv2.cvtColor(image,
cv2.COLOR_BGR2RGB)

# Kernel
kernel = np.ones((5,5), np.uint8)

# Erosion
eroded = cv2.erode(image_rgb, kernel,
iterations=1)

# Plot
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis('off')
plt.subplot(1,2,2)
plt.imshow(eroded)
```

```
plt.title("Eroded Image") plt.axis('off') plt.show()
# Save plt.imsave('eroded_output.png', eroded)
print("Eroded image saved successfully")
```



7.Perform basic video processing operations on the captured video. Read captured video in python and display the video, in slow motion and in fast motion.

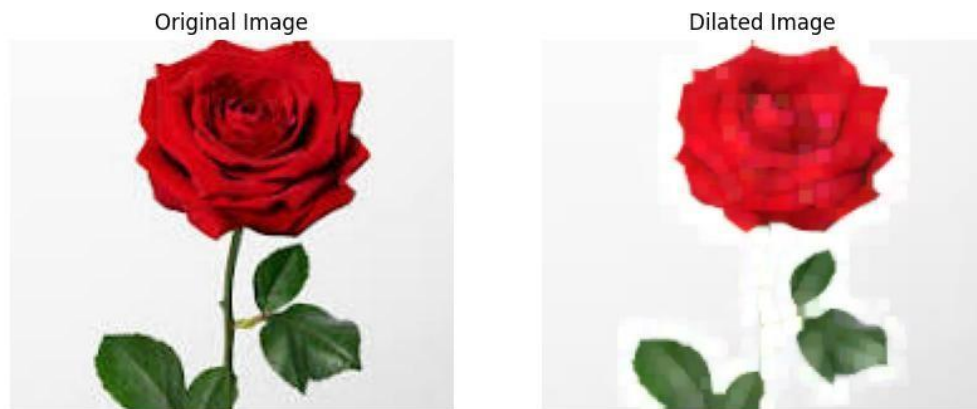
8.Perform basic Image Handling and processing operations on the image is to read an image in python and Dilate an Image using Dilate function

```
# Your Python code goes here
import cv2 import numpy as
np import matplotlib.pyplot as
plt
# Read image image = cv2.imread('image.jpg') # Convert
BGR to RGB image_rgb = cv2.cvtColor(image,
cv2.COLOR_BGR2RGB)
# Create kernel kernel =
np.ones((5,5), np.uint8)

# Apply dilation dilated = cv2.dilate(image_rgb,
kernel, iterations=1)
# Display
plt.figure(figsize=(10,4))
```



```
plt.subplot(1,2,1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis('off')
plt.subplot(1,2,2)
plt.imshow(dilated)
plt.title("Dilated Image")
plt.axis('off') plt.show()
# Save image plt.imsave("dilated_output.png",
dilated) print("Dilated image saved")
```



9.Implement the image scaling techniques to resize the images to both bigger and small sizes

```
# Image Scaling using OpenCV and Matplotlib import cv2 import matplotlib.pyplot as plt
# Read image image = cv2.imread('image.jpg') # Convert
BGR to RGB image_rgb = cv2.cvtColor(image,
cv2.COLOR_BGR2RGB)
# Resize Bigger bigger = cv2.resize(image_rgb,
None, fx=2, fy=2)
# Resize Smaller smaller = cv2.resize(image_rgb,
None, fx=0.5, fy=0.5)
```

```

# Print dimensions print("Original Image Size
:", image_rgb.shape) print("Bigger Image Size
:", bigger.shape) print("Smaller Image Size :",
smaller.shape)

# Display images
plt.figure(figsize=(15,5))

# Original plt.subplot(1,3,1) plt.imshow(image_rgb) plt.title(f'Original
Image\nSize: {image_rgb.shape[1]} x {image_rgb.shape[0]}')
plt.axis('off') # Bigger plt.subplot(1,3,2) plt.imshow(bigger)
plt.title(f'Bigger Image\nSize: {bigger.shape[1]} x {bigger.shape[0]}')
plt.axis('off') # Smaller plt.subplot(1,3,3) plt.imshow(smaller)
plt.title(f'Smaller Image\nSize: {smaller.shape[1]} x {smaller.shape[0]}')
plt.axis('off') plt.tight_layout() plt.show() # Save images
plt.imsave("bigger_image.png", bigger) plt.imsave("smaller_image.png",
smaller)

print("Images saved successfully")

```



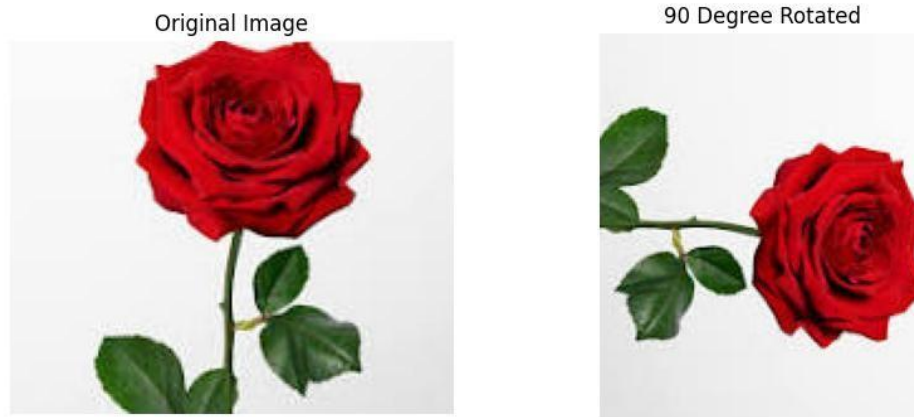
10. Perform a 90-degree rotation clockwise along the y-axis for the given image.

Your Python code goes here import cv2 import matplotlib.pyplot as plt image

```

= cv2.imread('image.jpg') image_rgb = cv2.cvtColor(image,
cv2.COLOR_BGR2RGB) rotated = cv2.rotate(image_rgb,
cv2.ROTATE_90_CLOCKWISE)
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis('off') plt.subplot(1,2,2)
plt.imshow(rotated) plt.title("90
Degree Rotated") plt.axis('off')
plt.show()
plt.imsave("rotate90.png", rotated)

```



11 .Perform a 180-degree rotation clockwise along the y-axis for the given image

```

# Your Python code goes here import cv2 import
matplotlib.pyplot as plt image = cv2.imread('image.jpg')
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
rotated = cv2.rotate(image_rgb, cv2.ROTATE_180)
plt.figure(figsize=(10,4)) plt.subplot(1,2,1)
plt.imshow(image_rgb) plt.title("Original Image")
plt.axis('off') plt.subplot(1,2,2) plt.imshow(rotated)

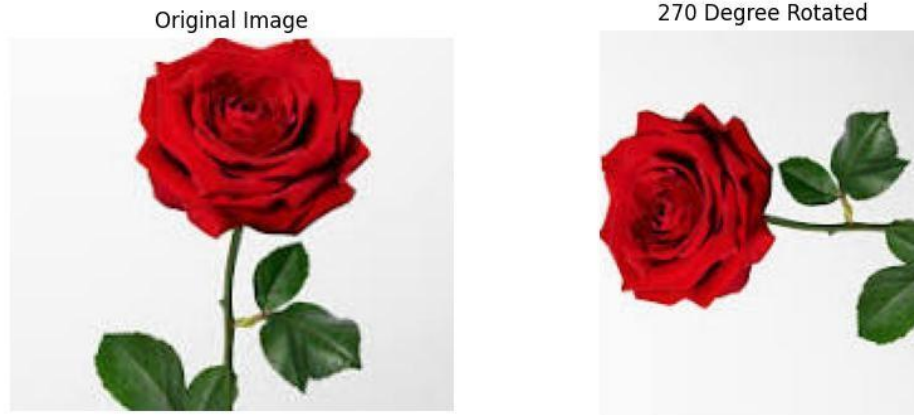
```

```
plt.title("180 Degree Rotated") plt.axis('off') plt.show()
plt.imsave("rotate180.png", rotated)
```



12. Perform a 270-degree rotation clockwise along the y-axis for the given image.

```
# Your Python code goes here
import cv2
import matplotlib.pyplot as plt
image = cv2.imread('image.jpg')
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
rotated = cv2.rotate(image_rgb, cv2.ROTATE_90_COUNTERCLOCKWISE)
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis('off')
plt.subplot(1,2,2)
plt.imshow(rotated)
plt.title("270 Degree Rotated")
plt.axis('off')
plt.show()
plt.imsave("rotate270.png", rotated)
```



13. Perform Affine Transformation on the given image using python and Open CV

```
# Your Python code goes here
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('image.jpg')
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
rows, cols = image_rgb.shape[:2]

# Points
pts1 = np.float32([[50,50],[200,50],[50,200]])
pts2 = np.float32([[10,100],[200,50],[100,250]])

# Affine matrix
M = cv2.getAffineTransform(pts1, pts2)

# Transform affine
image_affine = cv2.warpAffine(image_rgb, M, (cols, rows))

plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.imshow(image_rgb)
plt.title("Original Image")

plt.subplot(1,2,2)
plt.imshow(image_affine)
plt.title("Affine Transform")

plt.axis('off')
plt.show()

plt.imsave("affine_output.png", image_affine)
```

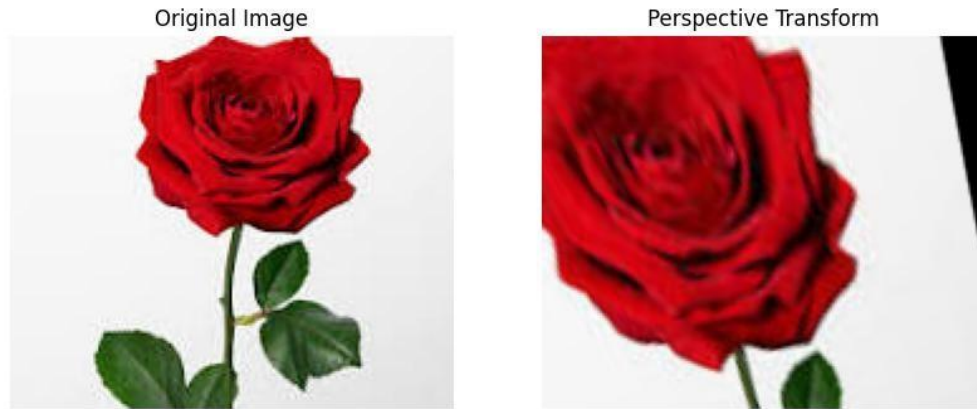


14. Perform Perspective Transformation on the given image using python and Open CV.

```
# Your Python code goes here
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('image.jpg')
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
rows, cols = image_rgb.shape[:2]
pts1 = np.float32([[50,50],[200,50],[50,200],[200,200]])
pts2 = np.float32([[10,100],[200,50],[100,250],[220,220]])
M = cv2.getPerspectiveTransform(pts1, pts2)

perspective = cv2.warpPerspective(image_rgb, M, (cols, rows))
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis('off')
plt.subplot(1,2,2)
plt.imshow(perspective)
plt.title("Perspective Transform")
plt.axis('off')
plt.show()
plt.imwrite("perspective_output.png", perspective)
```



15. Perform basic Image Handling and processing operations on the image is to read an image in python and detect the corners in the image using Harris Corner Detection function

Your Python code goes here

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
image = cv2.imread('image.png')
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
gray = np.float32(gray) # Harris Corner
Detection corners = cv2.cornerHarris(gray,
2, 3, 0.04)
# Mark corners image_rgb[corners > 0.01 *
corners.max()] = [255,0,0]
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis('off')
plt.subplot(1,2,2)
plt.imshow(image_rgb)
plt.title("Harris Corner Detection")
plt.axis('off')
```

```
plt.show() plt.imsave("harris_corner_output.png",  
image_rgb) print("Corner detected image saved")
```



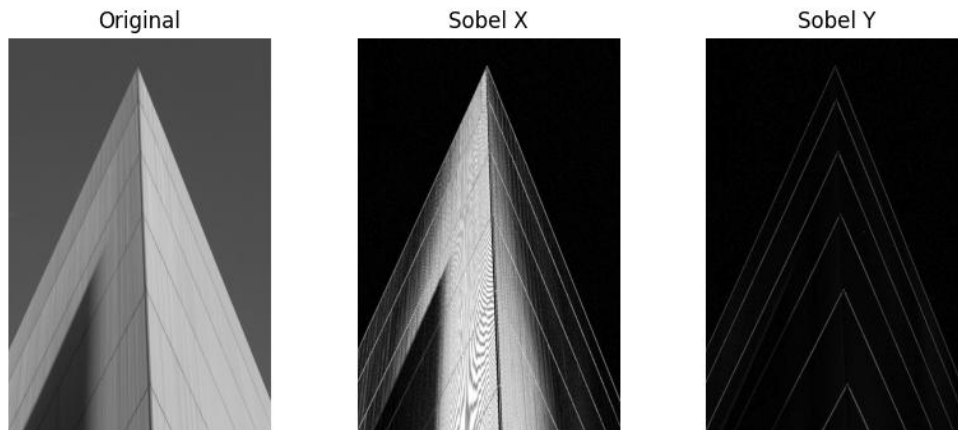
16. Implement a Sobel algorithm using Open CV to filter the input image.

Your Python code goes here

```
import cv2  
  
import matplotlib.pyplot as plt  
  
img = cv2.imread('image.jpg', 0)  
  
sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)  
sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)  
  
plt.figure(figsize=(10,4))  
  
plt.subplot(1,3,1)  
plt.imshow(img, cmap='gray')  
plt.title("Original")  
plt.axis('off')  
  
plt.subplot(1,3,2)  
plt.imshow(sobelx, cmap='gray')  
plt.title("Sobel X")  
plt.axis('off')  
  
plt.subplot(1,3,3)  
plt.imshow(sobely, cmap='gray')
```



```
plt.title("Sobel Y")
plt.axis('off')
plt.show()
```



17.Design and implement a water marking technique to insert the watermark into the original effectively image using Open CV.

```
import cv2

import matplotlib.pyplot as plt

# Read image
image = cv2.imread('image.jpg')

# Convert BGR to RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Get image dimensions
height, width, channels = image_rgb.shape

print("Image Width :", width)
print("Image Height :", height)

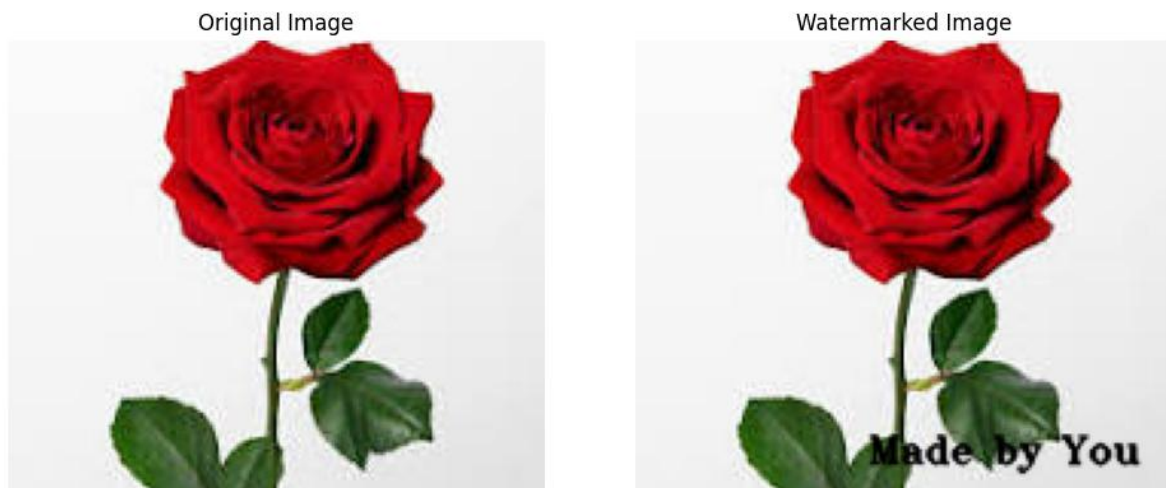
# Copy image
watermarked = image_rgb.copy()

# Watermark text
text = "Made by You"

# Font settings
font = cv2.FONT_HERSHEY_TRIPLEX
```

```
font_scale = 0.4
thickness = 1
# Get text size
(text_width, text_height), baseline = cv2.getTextSize(
    text,
    font,
    font_scale,
    thickness
)
# Bottom-right position calculation
x = width - text_width - 10
y = height - 10
print("Watermark Position :", x, y)
# Add watermark
cv2.putText(
    watermarked,
    text,
    (x, y),
    font,
    font_scale,
    (0, 0, 0),    # Black text
    thickness,
    cv2.LINE_AA
)
# Display images
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis('off')
```

```
plt.subplot(1,2,2)
plt.imshow(watermarked)
plt.title("Watermarked Image")
plt.axis('off')
plt.tight_layout()
plt.show()
# Save output
plt.imsave("watermarked_output.png", watermarked)
print("Watermarked image saved successfully")
```



18.Implement image cropping, copying and pasting to select a region of interest (ROI) from the source image using Open CV.

```
import cv2

import matplotlib.pyplot as plt

# Read image
image = cv2.imread('image.jpg')

# Convert BGR to RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Get image dimensions
height, width, _ = image_rgb.shape

print("Image Size:", width, "x", height)

# Crop ROI safely
```

```
roi = image_rgb[50:150, 50:150]
# Copy image
image_copy = image_rgb.copy()
# ROI dimensions
roi_h, roi_w, _ = roi.shape
# Safe paste position
x = width - roi_w - 10
y = height - roi_h - 10
# Paste ROI
image_copy[y:y+roi_h, x:x+roi_w] = roi
# Display
plt.figure(figsize=(12,4))
plt.subplot(1,3,1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis('off')
plt.subplot(1,3,2)
plt.imshow(roi)
plt.title("Cropped ROI")
plt.axis('off')
plt.subplot(1,3,3)
plt.imshow(image_copy)
plt.title("Copied & Pasted ROI")
plt.axis('off')
plt.tight_layout()
plt.show()
# Save output
plt.imsave("roi_output.png", image_copy)
print("ROI copied and saved successfully")
```



19. Implement the Erosion Morphological operations technique using Open CV in python.

Your Python code goes here

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image
image = cv2.imread('image.jpg')

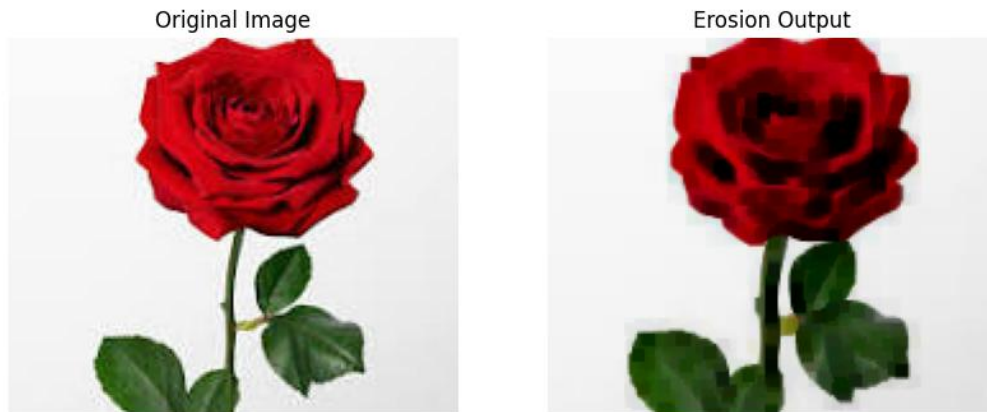
# Convert to RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Kernel
kernel = np.ones((5,5), np.uint8)

# Erosion
eroded = cv2.erode(image_rgb, kernel, iterations=1)

# Display
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.imshow(image_rgb)
plt.title("Original Image")
```

```
plt.axis('off')
plt.subplot(1,2,2)
plt.imshow(eroded)
plt.title("Erosion Output")
plt.axis('off')
plt.show()
# Save
plt.imsave("erosion_output.png", eroded)
print("Erosion image saved")
```



20. Implement the Dilation technique as a Morphological operation to dilate the foreground regions based on Open CV.

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

# Read image
image = cv2.imread('image.jpg')

# Convert to RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Kernel
kernel = np.ones((5,5), np.uint8)
```

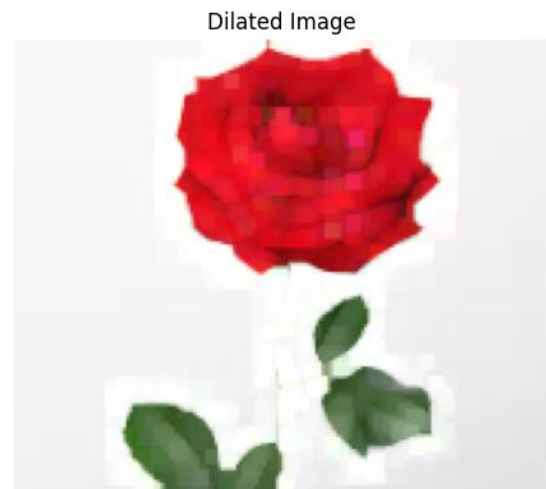
```

# Dilation
dilated = cv2.dilate(image_rgb, kernel, iterations=1)

# Display
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis('off')
plt.subplot(1,2,2)
plt.imshow(dilated)
plt.title("Dilated Image")
plt.axis('off')
plt.tight_layout()
plt.show()

# Save
plt.imsave("dilation_output.png", dilated)

```



21. Implement the Opening technique as a Morphological operation to dilate the foreground regions based on Open CV.

```

# Your Python code goes here
import cv2

```

```
import numpy as np
import matplotlib.pyplot as plt

# Read image
image = cv2.imread('image.jpg')

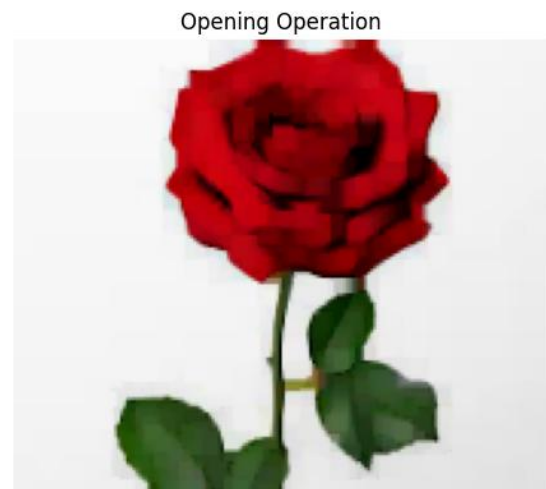
# Convert to RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Kernel
kernel = np.ones((5,5), np.uint8)

# Opening
opening = cv2.morphologyEx(image_rgb, cv2.MORPH_OPEN, kernel)

# Display
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis('off')
plt.subplot(1,2,2)
plt.imshow(opening)
plt.title("Opening Operation")
plt.axis('off')
plt.tight_layout()
plt.show()

# Save
plt.imsave("opening_output.png", opening)
```

22. Implement the Closing technique as a Morphological operation to dilate the foreground regions based on Open CV.

Your Python code goes here

```
import cv2
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Read image
```

```
image = cv2.imread('image.jpg')
```

```
# Convert to RGB
```

```
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

```
# Kernel
```

```
kernel = np.ones((5,5), np.uint8)
```

```
# Closing
```

```
closing = cv2.morphologyEx(image_rgb, cv2.MORPH_CLOSE, kernel)
```

```
# Display
```

```
plt.figure(figsize=(10,4))
```

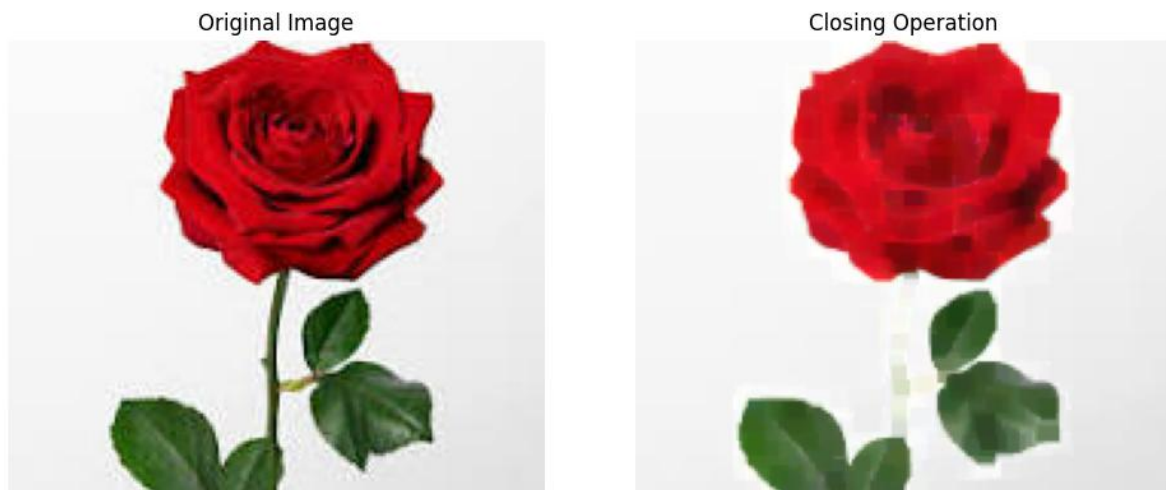
```
plt.subplot(1,2,1)
```

```
plt.imshow(image_rgb)
```

```
plt.title("Original Image")
```

```
plt.axis('off')
```

```
plt.subplot(1,2,2)
plt.imshow(closing)
plt.title("Closing Operation")
plt.axis('off')
plt.tight_layout()
plt.show()
# Save
plt.imsave("closing_output.png", closing)
```



23. Implement the Top hat technique as a Morphological operation to dilate the foreground regions based on Open CV.

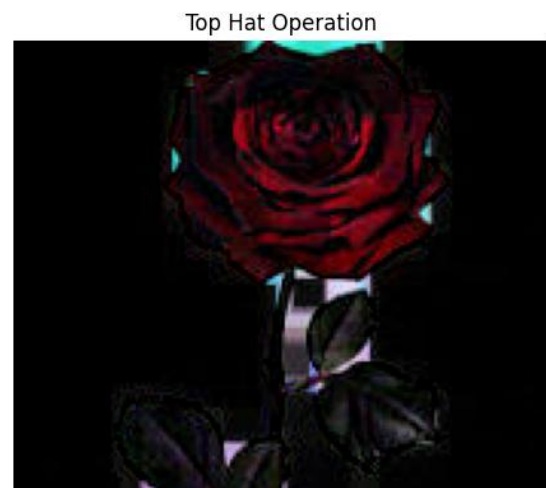
Your Python code goes here

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Read image
image = cv2.imread('image.jpg')
# Convert to RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
# Kernel
kernel = np.ones((9,9), np.uint8)
# Top Hat
```

```

tophat = cv2.morphologyEx(image_rgb, cv2.MORPH_TOPHAT, kernel)
# Display
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis('off')
plt.subplot(1,2,2)
plt.imshow(tophat)
plt.title("Top Hat Operation")
plt.axis('off')
plt.tight_layout()
plt.show()
# Save
plt.imsave("tophat_output.png", tophat)

```



24. Implement the Black hat technique as a Morphological operation to dilate the foreground regions based on Open CV.

Your Python code goes here

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

```

```
# Read image
image = cv2.imread('image.jpg')

# Convert to RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Kernel
kernel = np.ones((9,9), np.uint8)

# Black Hat
blackhat = cv2.morphologyEx(image_rgb, cv2.MORPH_BLACKHAT, kernel)

# Display
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis('off')
plt.subplot(1,2,2)
plt.imshow(blackhat)
plt.title("Black Hat Operation")
plt.axis('off')
plt.tight_layout()
plt.show()

# Save
plt.imwrite("blackhat_output.png", blackhat)
```



25. Recognize watch from the given image by general Object recognition using Open CV.

```
import cv2
import matplotlib.pyplot as plt
import numpy as np

# Read image
image = cv2.imread('watch.jpg')

# Convert BGR to RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Blur image
blur = cv2.GaussianBlur(gray, (9,9), 2)

# Detect circles
circles = cv2.HoughCircles(
    blur,
    cv2.HOUGH_GRADIENT,
    dp=1.2,
    minDist=300,
    param1=100,
    param2=50,
```

```

    minRadius=80,
    maxRadius=150
)
# Draw only largest circle
if circles is not None:
    circles = np.round(circles[0, :]).astype("int")
    # Select largest circle
    largest = max(circles, key=lambda c: c[2])
    x, y, r = largest
    cv2.circle(image_rgb, (x, y), r, (255,0,0), 4)
    cv2.putText(
        image_rgb,
        'Watch Detected',
        (x-60, y-r-15),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.8,
        (255,0,0),
        2
    )
# Display
plt.figure(figsize=(8,6))
plt.imshow(image_rgb)
plt.title("Watch Recognition")
plt.axis('off')
plt.show()
# Save output
plt.imsave("watch_detection_output.png", image_rgb)

```



26. Implement a function to reverse the frames of the video to create a video in reverse mode using Open CV.

Your Python code goes here

```
import cv2

cap = cv2.VideoCapture('video.mp4')

frames = []

# Read all frames
while True:
    ret, frame = cap.read()
    if not ret:
        break
    frames.append(frame)

cap.release()

# Reverse frames
frames.reverse()

# Video writer
height, width, _ = frames[0].shape
out = cv2.VideoWriter(
    'reverse_video.mp4',
```

```

cv2.VideoWriter_fourcc(*'mp4v'),
20,
(width, height)
)
# Save reverse video
for frame in frames:
    out.write(frame)
out.release()
print("Reverse video saved")

```

27. Implement a face detection algorithm using Open CV to detect and locate human faces in the images.

```

import cv2

import matplotlib.pyplot as plt

# Read image
image = cv2.imread('download.jpg')
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Load cascade
face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + 'haarcascade_frontalface_alt2.xml'
)

# Detect faces
faces = face_cascade.detectMultiScale(
    gray,
    scaleFactor=1.1,
    minNeighbors=6,
    minSize=(80,80)
)

# Draw rectangles
for (x, y, w, h) in faces:

```



```

cv2.rectangle(image_rgb, (x,y), (x+w,y+h), (255,0,0), 3)
# Display
plt.figure(figsize=(8,6))
plt.imshow(image_rgb)
plt.title("Face Detection")
plt.axis('off')
plt.show()
print("Faces detected:", len(faces))

```

Face Detection



28. Implement a vehicle detection algorithm using Open CV to detect and locate vehicles in each frame of the video.

```

import cv2

import matplotlib.pyplot as plt

# Read image
image = cv2.imread('image.jpg')

```

```
# Check image loaded
if image is None:
    print("Image not found")
    exit()

# Convert BGR to RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

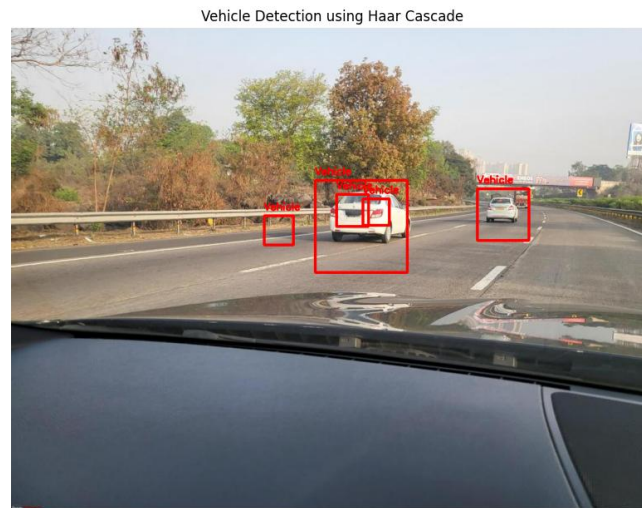
# Load vehicle cascade
car_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + 'haarcascade_car.xml'
)

# Check cascade loaded
if car_cascade.empty():
    print("XML file not loaded")
    exit()

# Detect vehicles
cars = car_cascade.detectMultiScale(
    gray,
    scaleFactor=1.1,
    minNeighbors=3,
    minSize=(40,40)
)

# Draw rectangles
for (x, y, w, h) in cars:
    cv2.rectangle(
        image_rgb,
        (x, y),
        (x + w, y + h),
```

```
(255, 0, 0),
3
)
cv2.putText(
    image_rgb,
    'Vehicle',
    (x, y - 10),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.6,
    (255, 0, 0),
    2
)
# Display
plt.figure(figsize=(10,6))
plt.imshow(image_rgb)
plt.title("Vehicle Detection using Haar Cascade")
plt.axis('off')
plt.show()
# Save output
plt.imsave("vehicle_detection_output.png", image_rgb)
print("Vehicle detection completed")
```



29. Implement an Eye detection algorithm using Open CV to detect and locate human eyes in the images.

Your Python code goes here

```
import cv2

import matplotlib.pyplot as plt

# Read image
image = cv2.imread('image.jpg')

image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

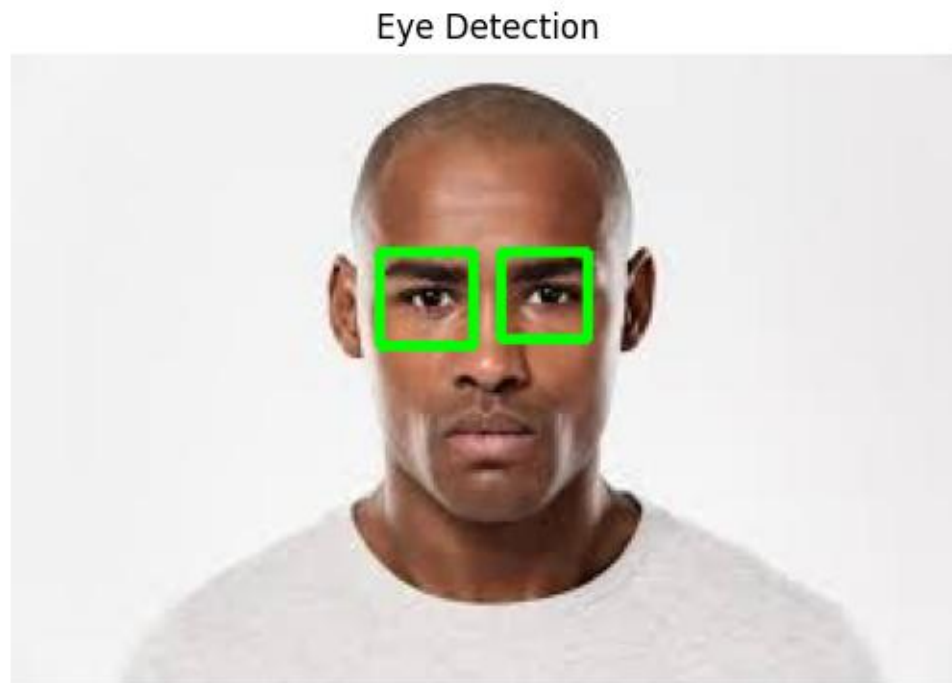
# Eye Cascade
eye_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + 'haarcascade_eye.xml'
)

# Detect eyes
eyes = eye_cascade.detectMultiScale(gray, 1.1, 4)

# Draw rectangles
for (x,y,w,h) in eyes:
    cv2.rectangle(image_rgb, (x,y), (x+w,y+h), (0,255,0), 2)

# Display
```

```
plt.imshow(image_rgb)
plt.title("Eye Detection")
plt.axis('off')
plt.show()
```



30. Implement a Smile detection algorithm using Open CV to detect and locate human smile in the images.

Your Python code goes here

```
import cv2
```

```
import matplotlib.pyplot as plt
```

```
# Read image
```

```
image = cv2.imread('image.jpg')
```

```
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
# Smile Cascade
```

```

smile_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + 'haarcascade_smile.xml'
)

# Detect smiles
smiles = smile_cascade.detectMultiScale(gray, 1.8, 20)

# Draw rectangles
for (x,y,w,h) in smiles:
    cv2.rectangle(image_rgb, (x,y), (x+w,y+h), (255,0,0), 2)

# Display
plt.imshow(image_rgb)
plt.title("Smile Detection")
plt.axis('off')
plt.show()

```

Smile Detection



31. Implement a Segmentation algorithm using Open CV to segment the given input image based on the given threshold values.

32. Write a Python function to create a white image size entered by the user and then create 4 boxes of Black, Blue, Green and Red respectively on each corner of the image. The size of the colored

boxes should be 1/10th the size of the image. (HINT: the arrays of ones and zeros can be in more than 2 dimensions).

33. Write a Python function to create a white image size entered by the user and then create a shape of Rectangle using Open CV.

34. Write a Python function to create a white image size entered by the user and then create a shape of Circle using Open CV.

35. Write a Python function to create a text string entered by the user that must be appeared on the given image using Open CV.

36. Write a Python function to subtract the background of the given input image based on color levels using Open CV

37. Write a Python function to subtract the foreground of the given input image based on color levels using Open CV.

38. Write a Python function to Count the number of faces for the given input image using Open CV.

39. Write a Python function to play the given video in reverse mode in slow motion.

40. Write a Python function to extract the text from videos.